MySQL 中的日志类型有哪些？binlog、redo log 和 undo log 的作用和区别是什么？ - 面试鸭 | 2026最新面试题+详细答案解析

> ## Excerpt
>
> 2026最新MySQL面试题，包含详细答案解析和练习题，8613+程序员已学习，已帮助10万+程序员通过校招/社招/实习面试，拿到offer！

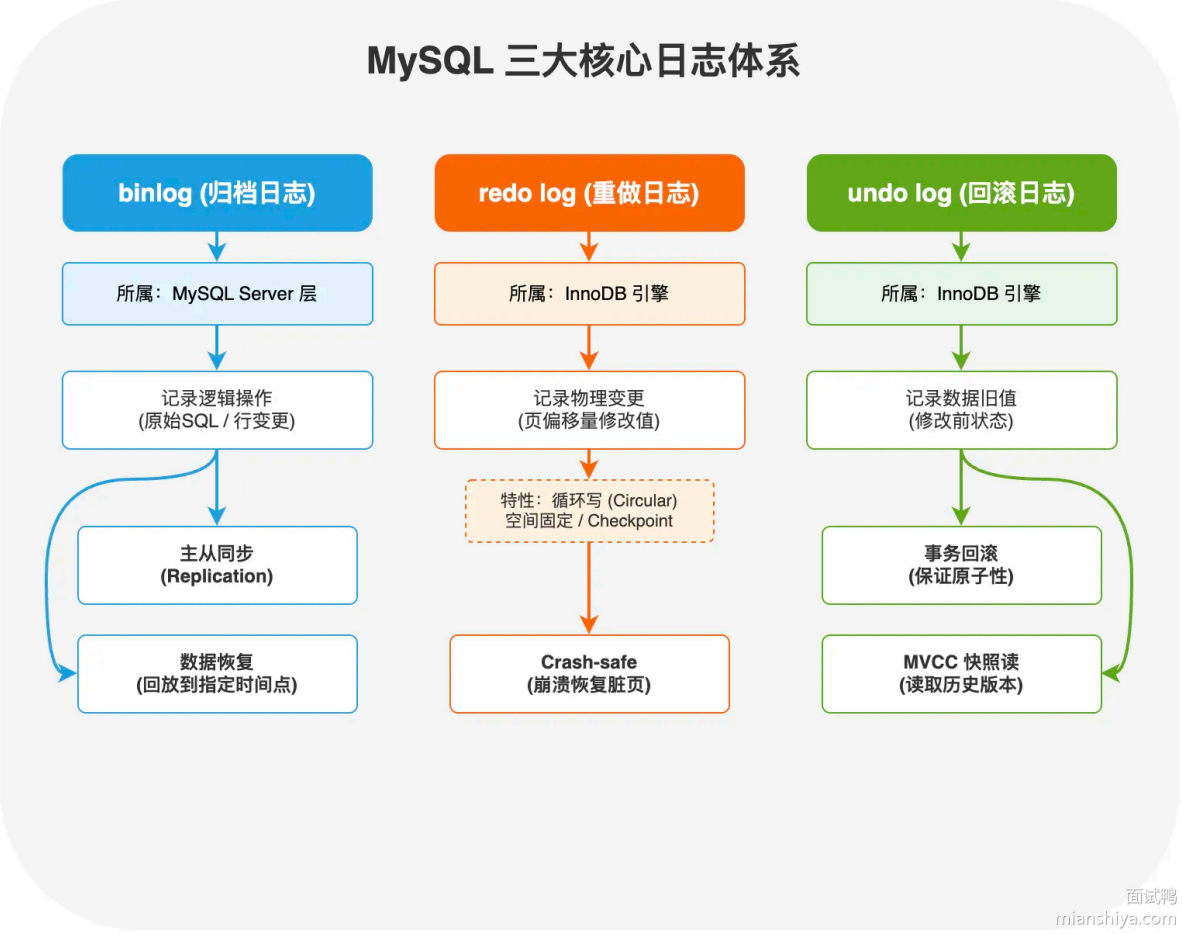
回答重点

MySQL 有三种核心日志，各司其职：**binlog** 负责主从复制和数据恢复，**redo log** 保证崩溃后数据不丢，**undo log** 支撑事务回滚和 MVCC。

1）binlog 是 Server 层的日志，记录的是逻辑操作，也就是原始 SQL 或者行变更前后的值。它的核心场景是主从同步，从库拉取主库的 binlog 重放一遍就能保持数据一致。另外做数据恢复的时候，也是靠 binlog 配合全量备份回放到指定时间点。

2）redo log 是 InnoDB 引擎独有的，记录的是物理变更，具体就是"某个数据页的某个偏移量改成了什么值"。它的作用是 crash-safe，MySQL 挂了重启后，InnoDB 会用 redo log 把没来得及刷盘的脏页恢复出来。redo log 是循环写的，空间固定，写满了就得等 checkpoint 推进才能继续。

3）undo log 也是 InnoDB 的，记录的是数据修改前的旧值。事务回滚的时候，就靠 undo log 把数据改回去。另外 MVCC 的快照读也依赖它，别的事务要读历史版本，顺着 undo log 链往前找就行。



三者的本质区别：binlog 记录的是数据变更的逻辑语义，由 Server 层生成，跨引擎通用。redo log 和 undo log 服务于 InnoDB 的事务与恢复机制，强依赖引擎内部实现，只有 InnoDB 才有。binlog 可以无限追加，redo log 是循环覆盖写。

扩展知识

redo log 的设计动机

InnoDB 用 Buffer Pool 缓存数据页，修改操作先改内存里的脏页，然后异步刷盘。问题来了，如果脏页还没刷盘 MySQL 就崩了，数据不就丢了？

最简单的方案是每次修改都立刻刷盘，但数据页是 16KB，改一个字节就要写 16KB，而且数据页在磁盘上是随机分布的，随机 IO 性能很差，一台普通 SSD 每秒也就几千次随机写。

redo log 的思路是**先写日志再写数据页**。redo log 是顺序追加写的，几十个字节就能记录一次修改，顺序 IO 性能比随机 IO 高一个数量级。只要 redo log 落盘了，就算数据页没刷，重启后也能恢复。这就是经典的 **WAL**，Write-Ahead Logging。

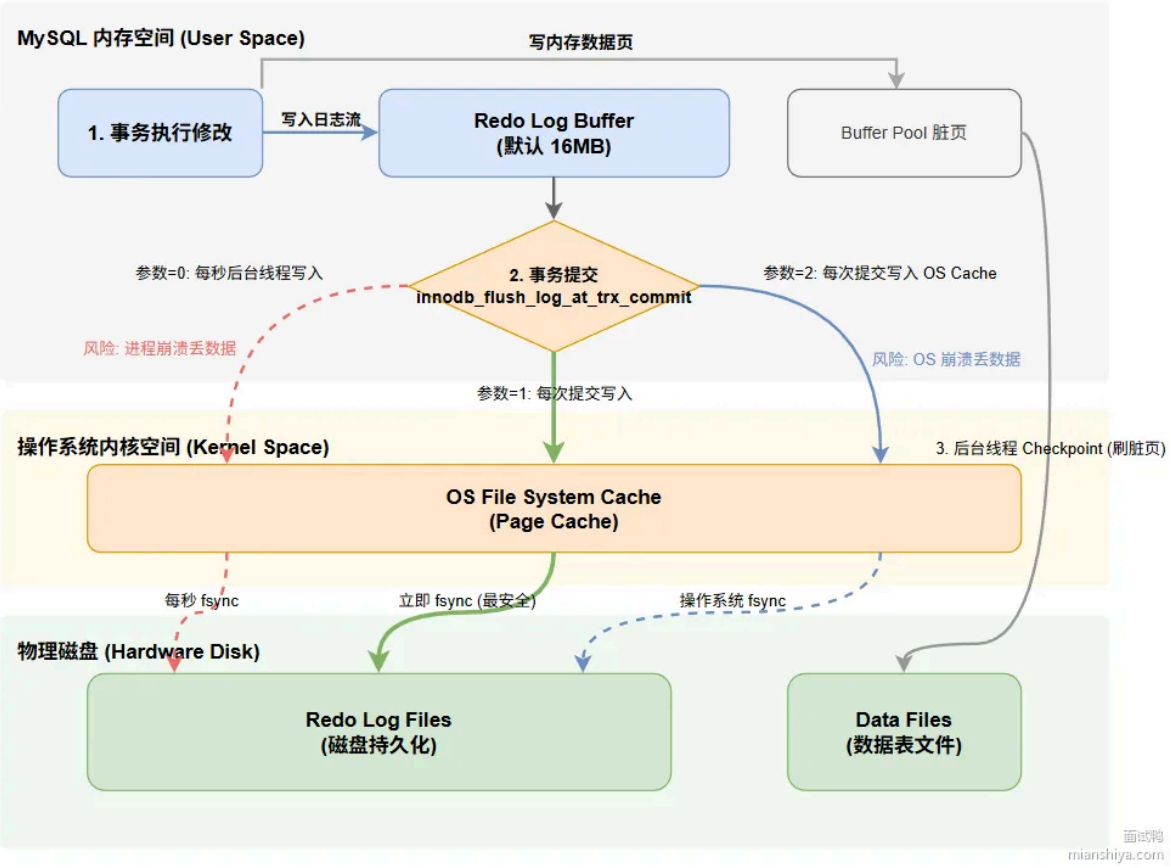
redo log 的写入流程分三步：

1）事务执行过程中，先把修改写到 redo log buffer，这块内存默认 16MB

2）事务提交时，根据 innodb_flush_log_at_trx_commit 参数决定刷盘策略：

- 设为 0，每秒刷一次，可能丢 1 秒数据
- 设为 1，每次提交都刷盘，最安全但性能最差
- 设为 2，每次提交写到 OS 缓存，MySQL 挂了不丢数据，操作系统挂了可能丢

3）后台线程定期把脏页刷到磁盘，推进 checkpoint



binlog 和 redo log 的两阶段提交

一个事务提交，binlog 和 redo log 都要写，那谁先谁后？如果只写了一个另一个没写成功，主从数据就不一致了。

InnoDB 用两阶段提交来解决：

1）prepare 阶段：redo log 写入并标记为 prepare 状态

2）commit 阶段：binlog 写入成功后，再把 redo log 标记为 commit

崩溃恢复时，如果 redo log 是 prepare 状态，就去看 binlog 有没有对应的记录。有就提交，没有就回滚。这样保证了两份日志的一致性。

详细看：[9498. MySQL 事务的二阶段提交是什么？]

undo log 与 MVCC

undo log 除了支持回滚，更重要的是支撑 MVCC。InnoDB 的每一行数据都有两个隐藏字段：trx_id 记录最后修改这行的事务 ID，roll_pointer 指向 undo log 里的上一个版本。

当一个事务执行快照读的时候，会生成一个 Read View，里面记录了当前活跃的事务 ID 列表。读数据时顺着版本链往前找，找到第一个对当前事务可见的版本返回。这就是为什么 RR 隔离级别下，一个事务里多次读同一行数据结果一样，因为 Read View 是事务开始时就定死了。

三种日志对比

维度	binlog	redo log	undo log
所属层级	Server 层	InnoDB 引擎	InnoDB 引擎
日志类型	逻辑日志	物理日志	逻辑日志
写入方式	追加写，文件无限增长	循环写，固定大小	链式存储
核心作用	主从复制、数据恢复	崩溃恢复	事务回滚、MVCC
事务相关	事务提交后写入	事务执行中持续写入	事务执行中写入

生产环境配置建议

对于核心业务库，innodb_flush_log_at_trx_commit 和 sync_binlog 都建议设成 1，也就是双 1 配置，保证数据绝对不丢。代价是每次提交都要两次刷盘，TPS 会下降不少，一般在几千到一万左右。

如果业务能容忍少量数据丢失，比如日志库、监控库，可以把这俩参数设成 0 或 2，性能能提升好几倍。

面试官追问

提问：redo log 写满了会怎么样？

回答：MySQL 会阻塞所有更新操作，强制推进 checkpoint，把一部分脏页刷到磁盘，腾出空间后才能继续写。这时候数据库会出现明显的抖动，TPS 断崖式下降。生产环境要监控 redo log 的使用率，发现经常接近写满就得调大 innodb_log_file_size，比如从默认的 48MB 调到 1-2GB。

提问：binlog 有哪几种格式？各有什么优缺点？

回答：三种格式： 1）statement 格式记录原始 SQL，日志量小但可能导致主从不一致，比如 now() 函数在主从执行时间不同 2）row 格式记录行变更前后的完整数据，日志量大但绝对安全，生产环境基本都用这个 3）mixed 格式自动判断，普通语句用 statement，有风险的用 row，实际很少用因为判断逻辑不够智能

提问：为什么 redo log 要设计成循环写？直接追加写不行吗？

回答：追加写意味着空间无限增长，磁盘迟早撑爆。redo log 的目的只是保证 crash-safe，只要脏页刷盘了，对应的 redo log 就没用了，所以设计成固定大小循环覆盖写。checkpoint 往前推进就是在标记哪些 redo log 已经没用了、可以被覆盖。binlog 之所以追加写，是因为它还要用来做主从复制和历史数据恢复，得长期保留。

提问：undo log 什么时候会被清理？

回答：当没有任何事务需要用到这个版本的 undo log 时才能清理。具体来说，就是所有活跃事务的 Read View 都不需要看这个旧版本了。InnoDB 有个 purge 线程专门干这事，它会定期扫描，把没人用的 undo log 回收掉。如果有长事务一直不提交，undo log 就会堆积，回滚段撑爆，这就是为什么要避免大事务和长事务。